

Presentation & viva

Student Attendance & Performance Tracker

A 7-minute talk, a safe demo order, and concise answers for tomorrow.

7-minute presentation

0:00-0:45 - Introduce the problem

Good morning. My project is the Student Attendance and Performance Tracker. It brings student records, subject enrollment, attendance, and assessment results into one Android application. The aim is to reduce repeated manual entry and give administrators, faculty, and students different views of the same academic information.

0:45-1:30 - Explain the architecture

The Android application is written in Kotlin using Jetpack Compose. Compose builds the interface from functions and updates it when state changes. Retrofit and OkHttp handle communication with the backend. Google Apps Script implements the server logic, and Google Sheets stores the records. Google Sheets and Apps Script are required parts of this project.

The flow is: screen, API client, Apps Script, Google Sheets, and then a JSON response back to the screen. The server checks permissions before reading or changing records.

1:30-2:15 - Explain login and roles

The login screen requests a Google ID token. Apps Script verifies its signature and checks its issuer, audience, and expiry. It then looks up the account in the TrackerUsers sheet. This sheet decides whether the account is an administrator, faculty member, or student. Selecting a screen on the phone does not grant permission; the backend also checks the role.

Tokens stay in memory. When the session expires or the user signs out, the app returns to login. The current build requires the institution's Apps Script deployment URL and Google web client ID before real login can work.

2:15-3:00 - Administrator workflow

An administrator can manage student, faculty, and subject profiles. They can assign faculty to subjects and enroll students into a subject. The dashboard shows counts from the backend. Deactivation preserves previous academic history rather than deleting it. Records have revision numbers so one user's old form cannot silently overwrite another user's newer change.

3:00-4:00 - Faculty workflow

A faculty member sees their assigned subjects. For attendance, they select a date and class slot and mark each enrolled student present, absent, late, or excused. The backend requires a complete roster and prevents duplicate submission of the same class session. Present and late count as attended; excused records are excluded from the denominator.

Faculty can also create assessments, save draft marks, and publish results. A blank mark is pending, zero is a valid score, and absent is a separate status. Publishing requires every student to have a result. Published assessments are locked in this version.

4:00-4:40 - Student workflow

A student sees their own attendance summary and their own published results. Draft assessments are hidden. The server derives the student identity from the verified account, so changing a student ID in a request does not expose

another student's data. When there are no counted attendance sessions, the app shows that there is no percentage yet.

4:40-5:25 - Reports and data storage

The attendance report can be shared as a CSV file. The exporter handles commas, quotes, and spreadsheet formula prefixes. Android shares only the generated export through a restricted FileProvider.

The backend uses two sheets. TrackerUsers stores approved accounts and roles. TrackerRecords stores an append-only journal with the record type, identifier, JSON data, time, actor, and action. The latest entry for an identifier is its current state. Script locking protects operations from conflicting concurrent writes.

5:25-6:15 - Show important code

I would start with MainActivity, which installs the Compose interface and observes the session. AppNavigation chooses the role's screens. GoogleSheetsApi defines the request and response models. TrackerInterceptor changes requests into authenticated JSON POST requests. In the backend, doPost verifies the identity, obtains a lock, and calls createService. createService contains the actual permission and academic rules.

For example, when a teacher saves attendance, AttendanceScreen builds the roster entries, the API sends them to Apps Script, createService validates the subject and roster, and the sheet store appends the accepted record.

6:15-7:00 - Validation and conclusion

The Android debug build, local unit tests, and lint checks pass. The backend has 27 passing tests covering identity verification, permissions, attendance, assessment rules, and the Apps Script adapter. Lint still reports warnings. Device instrumentation and live institution deployment have not been verified here.

The implemented code covers the core role-based academic workflows. The next practical step is to configure Google authentication, deploy Apps Script, populate approved accounts, and run the live acceptance checklist. Larger production improvements include monitoring, backup and restore procedures, pagination, offline support, and a controlled correction process for published records. Thank you.

Demo sequence

1. Verify Google login and deployment in advance. Use real approved test accounts only.
2. Administrator: open student, faculty, and subject directories; show one subject's enrollment.
3. Faculty: open assigned subject; fill one attendance session; confirm submission; view report.
4. Faculty: create an assessment draft; demonstrate zero versus pending; complete marks and publish.
5. Student: open own attendance and published results.
6. Faculty: share a CSV report.
7. If deployment is unavailable, present the source flow and test reports. Do not describe a source walkthrough as a working live demo.

Short answers for likely questions

- Why Compose? It uses Kotlin functions and observable state to describe the UI, which makes role screens and loading states easier to compose.
- Why Sheets? It is the required storage platform and makes institution data inspectable. Apps Script supplies the API and permission checks. Its quotas and full-journal reads limit large-scale use.
- Is the app completely production ready? The core implementation is locally validated. Live deployment, device acceptance, operational monitoring, backups, and remaining production work still need verification.

- Who assigns roles? An administrator maintains the approved TrackerUsers sheet. The Android app cannot assign itself a role.
- How is attendance calculated? Present plus late, divided by present plus late plus absent, multiplied by 100. Excused records do not count. No counted sessions means no percentage.
- Why use a revision? To reject stale edits and prevent accidental overwrites.
- Why an append-only journal? It retains the actor, action, and previous record versions. The latest version is used for normal reads.
- How are repeated requests handled? Attendance identifies a class session by subject, date, and slot; request identifiers and matching payloads handle supported retries. This is not a durable offline queue.
- Can a student read someone else's marks? The server scopes the response to the verified student's profile and includes only published assessments.
- Can published marks be edited? Not in the current workflow. A future correction process should explicitly preserve an audit history.
- What is a coroutine? A Kotlin mechanism for suspending asynchronous work without blocking the UI thread.
- What is rememberSaveable? Compose state that can survive supported activity recreation; it is not backend storage or a guaranteed offline database.
- What is Retrofit? A library that turns a Kotlin interface into HTTP calls and converts JSON using Gson.
- What does FileProvider do? It gives another app temporary access to the generated export without exposing the entire cache directory.

Tonight's rehearsal order

Read MainActivity, AppNavigation, LoginScreen, NetworkModule, GoogleSheetsApi, AttendanceScreen, AssessmentsScreen, core.js, auth.js, and apps-script.js first. Rehearse one attendance request end to end. Then read the form screens and tests. Use the detailed PDF's file index and numbered listings to locate any remaining code.

Do not memorize every line. Explain each file's responsibility, its inputs, its output, and the rules it protects. Keep this script beside the detailed guide for rehearsal.